



Real-Time Physics on the Web

Bringing CAD Models to Life with Smart Math Compression

M2 Thesis Defense (IGA & ROM for Parameterized Dynamics)

Mumen Wehbe

Supervisor: Christophe Hoareau

Conservatoire National des Arts et Métiers

July 2026

Academic Context & Pedagogical Team

The Institution

Conservatoire National des Arts et Métiers (Cnam)

- Located in Paris, France.
- Dedicated to higher education, professional training, and scientific research.
- Specialized computational and research facilities.

Academic Framework

- **Program:** Master 2 (M2) Thesis.
- **Subject:** IGA & ROM for Parameterized Structural Dynamics.

Pedagogical & Research Team

- **Candidate:**
Mumen Wehbe
- **Academic Supervisor:**
Christophe Hoareau
- **Cnam Pedagogical Team:**
Academic coordinators, supervisors, and M2 thesis jury members.

Lecture Overview

1. Introduction & State of the Art
2. Theory: Problem Definition
3. Discretization: Isogeometric Analysis
4. Simulation: Full Order Model
5. Reduced Order Modeling, Hyper-Reduction & Digital Twin
6. Conclusion & Future Perspectives

CHAPTER 1

Introduction & State of the Art

Introduction & Motivation

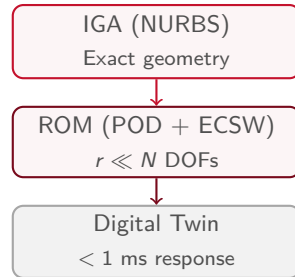
Pedagogical Origin to Research Level

This project began as a **pedagogical application** to demonstrate structural dynamics. However, to meet industry demands for interactive simulations, it has been elevated to a rigorous **research-level** implementation.

Objective & The Problem

How to achieve real-time, interactive parameter updates for complex shapes? Standard FEM requires slow online remeshing.

- **Our Solution:** Exact geometry (IGA) + Fast solvers (ROM).



Open Web Application

Conceptual Parallels to Data Compression

IGA

[Hughes et al., 2005]

- Exact CAD geometry (NURBS) — smooth boundaries without remeshing.

ROM

[Benner et al., 2015]

- Projects dynamics onto dominant energy-carrying modes (like lossy compression).
- Solves equations in reduced space in milliseconds.

Hyper-Reduction

[Farhat et al., 2014]

- Isolates a sparse set of active integration cells (like optimal sensor placement).
- Bypasses full-mesh assembly to resolve the online bottleneck.

CHAPTER 2

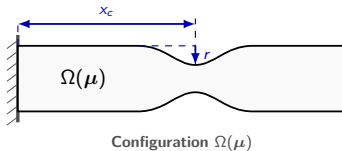
Theory: Problem Definition

Geometry Configurations

We consider a 2D cantilever beam with **symmetric smooth notches** parameterized by:

$$\mu = (r, x_c)^T \in \mathcal{D} \subset \mathbb{R}^2$$

- r — notch radius; x_c — notch center position
- $\mathcal{D} = [r_{\min}, r_{\max}] \times [x_{c,\min}, x_{c,\max}]$ — compact parameter domain



Governing Equations & Strong Form

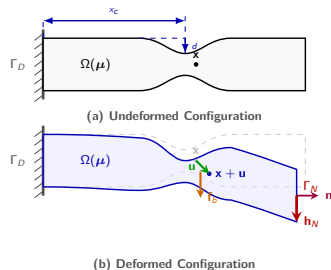
Assuming small strains and a linear elastic isotropic material, the balance of linear momentum on $\Omega(\mu) \times (0, T]$ is:

Strong Form

$$\rho \ddot{\mathbf{u}} - \nabla \cdot \boldsymbol{\sigma}(\mathbf{u}) = \mathbf{f}_b \quad \text{in } \Omega(\mu) \times (0, T]$$

Boundary, Initial & Constitutive Relations:

- $\mathbf{u} = \mathbf{0}$ on $\Gamma_D(\mu)$ (Clamped BC)
- $\boldsymbol{\sigma} \cdot \mathbf{n} = \mathbf{h}_N$ on $\Gamma_N(\mu)$ (Traction BC)
- $\mathbf{u} = \mathbf{u}_0, \dot{\mathbf{u}} = \mathbf{v}_0$ at $t = 0$ (Initial Conditions)
- $\boldsymbol{\sigma} = \mathbf{D} : \boldsymbol{\varepsilon}(\mathbf{u})$ (Hooke's Law)
- $\boldsymbol{\varepsilon}(\mathbf{u}) = \frac{1}{2}(\nabla \mathbf{u} + (\nabla \mathbf{u})^T)$ (Strain tensor)



Where: ρ — density; $\mathbf{f}_b$ — body force; $\mathbf{n}$ — normal; $\mathbf{D}$ — elasticity tensor

Classical Weak Form on $\Omega(\mu)$

Multiplying the strong form by test functions and integrating by parts yields:

Weak Formulation on the Physical Domain

Find $\mathbf{u}(t) \in \mathcal{V}(\Omega(\mu))$ such that:

$$m(\ddot{\mathbf{u}}, \mathbf{v}; \mu) + k(\mathbf{u}, \mathbf{v}; \mu) = f(\mathbf{v}; \mu) \quad \forall \mathbf{v} \in \mathcal{V}_0(\Omega(\mu))$$

$$\text{where: } m(\ddot{\mathbf{u}}, \mathbf{v}; \mu) = \int_{\Omega(\mu)} \rho \ddot{\mathbf{u}} \cdot \mathbf{v} \, d\Omega, \quad k(\mathbf{u}, \mathbf{v}; \mu) = \int_{\Omega(\mu)} \boldsymbol{\varepsilon}(\mathbf{v})^T \mathbf{D} \boldsymbol{\varepsilon}(\mathbf{u}) \, d\Omega$$

$$f(\mathbf{v}; \mu) = \int_{\Omega(\mu)} \mathbf{f}_b \cdot \mathbf{v} \, d\Omega + \int_{\Gamma_N(\mu)} \mathbf{h}_N \cdot \mathbf{v} \, d\Gamma$$

The Parameter Dependency Problem

Space Incompatibility: Both the domain $\Omega(\mu)$ and spaces $\mathcal{V}(\Omega(\mu))$ depend on the parameter μ .

Comparison Issue: Solutions at different parameters cannot be directly compared or linearly combined.

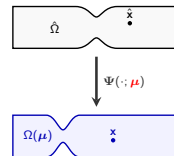
Pullback Mapping to Reference Configuration

To resolve space incompatibility, we map $\Omega(\boldsymbol{\mu})$ to a fixed reference domain $\hat{\Omega}$ via a geometric diffeomorphism:

$$\mathbf{x} = \Psi(\hat{\mathbf{x}}; \boldsymbol{\mu}), \quad \mathbf{J}_{\hat{\mathbf{x}}}(\hat{\mathbf{x}}; \boldsymbol{\mu}) = \frac{\partial \Psi}{\partial \hat{\mathbf{x}}}, \quad J_{\hat{\mathbf{x}}, \boldsymbol{\mu}} = \det \mathbf{J}_{\hat{\mathbf{x}}} > 0$$

Main Steps for the Pullback:

1. **Map functions:** Define $\hat{\mathbf{u}}(\hat{\mathbf{x}}) = \mathbf{u}(\Psi(\hat{\mathbf{x}}; \boldsymbol{\mu}))$ in parameter-independent space $\mathcal{V}(\hat{\Omega})$.
2. **Transform gradients:** $\nabla_{\mathbf{x}} \mathbf{u} = \nabla_{\hat{\mathbf{x}}} \hat{\mathbf{u}} \mathbf{J}_{\hat{\mathbf{x}}}^{-1}(\hat{\mathbf{x}}; \boldsymbol{\mu})$ (chain rule).
3. **Change integration variables:** $d\Omega = J_{\hat{\mathbf{x}}, \boldsymbol{\mu}} d\hat{\Omega}$ and $d\Gamma = J_{\hat{\mathbf{x}}, \boldsymbol{\mu}}^{\Gamma} d\hat{\Gamma}$.



Pulled-Back Weak Form on $\hat{\Omega}$

Applying the pullback steps to each term in the classical weak form yields:

Classical Form (Physical Domain $\Omega(\mu)$)

Integrals depend implicitly on μ via the shifting integration domain:

$$m(\ddot{\mathbf{u}}, \mathbf{v}; \mu) = \int_{\Omega(\mu)} \rho \ddot{\mathbf{u}} \cdot \mathbf{v} \, d\Omega$$

$$f(\mathbf{v}; \mu) = \int_{\Omega(\mu)} \mathbf{f}_b \cdot \mathbf{v} \, d\Omega + \int_{\Gamma_N(\mu)} \mathbf{h}_N \cdot \mathbf{v} \, d\Gamma$$

$$k(\mathbf{u}, \mathbf{v}; \mu) = \int_{\Omega(\mu)} \boldsymbol{\varepsilon}(\mathbf{v})^T \mathbf{D} \boldsymbol{\varepsilon}(\mathbf{u}) \, d\Omega$$

Limitation: Requires re-meshing and re-integration over a moving domain for every parameter change.

Pulled-Back Form (Reference Domain $\hat{\Omega}$)

Dependency on μ is isolated inside the Jacobian mapping terms:

$$\hat{m}(\ddot{\mathbf{u}}, \delta \mathbf{u}; \mu) = \int_{\hat{\Omega}} \rho \ddot{\mathbf{u}} \cdot \delta \mathbf{u} \, J_{\hat{\mathbf{x}}, \mu} \, d\hat{\Omega}$$

$$\hat{f}(\delta \mathbf{u}; \mu) = \int_{\hat{\Omega}} \mathbf{f}_b \cdot \delta \mathbf{u} \, J_{\hat{\mathbf{x}}, \mu} \, d\hat{\Omega} + \int_{\hat{\Gamma}_N} \mathbf{h}_N \cdot \delta \mathbf{u} \, J_{\hat{\mathbf{x}}, \mu}^{\Gamma} \, d\hat{\Gamma}$$

$$\hat{k}(\mathbf{u}, \delta \mathbf{u}; \mu) = \int_{\hat{\Omega}} (\mathbf{J}_{\hat{\mathbf{x}}}^{-T} \nabla_{\hat{\mathbf{x}}} \mathbf{u}) : \mathbf{D} : (\mathbf{J}_{\hat{\mathbf{x}}}^{-T} \nabla_{\hat{\mathbf{x}}} \delta \mathbf{u}) \, J_{\hat{\mathbf{x}}, \mu} \, d\hat{\Omega}$$

Advantage: Integration is performed over a static domain; variables are in a parameter-independent space.

CHAPTER 3

Discretization: Isogeometric Analysis

Curve Representations in Geometry

Comparing representations of a geometric curve $\mathbf{C}$:

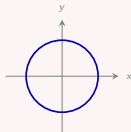
Explicit



$$y = x^2$$

Easy to compute, but cannot represent vertical lines or multiple-valued curves (e.g., circles).

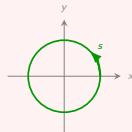
Implicit



$$x^2 + y^2 - R^2 = 0$$

Can represent arbitrary shapes/topologies, but hard to parameterize or evaluate coordinates directly.

Parametric



$$\begin{cases} x(s) = R \cos(s) \\ y(s) = R \sin(s) \end{cases}$$

Standard in CAD. Easy to generate coordinates, trace boundaries, and map variables. Flexible dimension scaling.

Why IGA?

IGA bridges the gap between CAD and FEA by using CAD basis functions for analysis:

- **Direct CAD Integration:** Runs simulations directly on NURBS geometry, bypassing time-consuming mesh generation.
- **Exact Geometry:** Preserves exact CAD boundaries throughout the analysis, eliminating geometric errors.
- **Higher Continuity (C^{p-1}):** Provides high regularity across element boundaries for superior accuracy per DOF compared to classical C^0 FEM.

The Core Ingredients of IGA

1. Knot Vector

Ordered set of parametric coordinates:

$$\Xi = \{\xi_0, \xi_1, \dots, \xi_m\}$$

Non-zero knot spans partition the parametric domain into elements.

2. B-Spline Basis

Polynomial basis functions constructed recursively over Ξ via Cox-de Boor:

$$N_{i,0}(\xi) = \begin{cases} 1 & \xi_i \leq \xi < \xi_{i+1} \\ 0 & \text{otherwise} \end{cases}$$

Yields C^{p-1} continuity across internal element boundaries.

3. Weights (NURBS)

Weights w_i rationalise the basis functions to construct NURBS:

$$R_{i,p}(\xi) = \frac{N_{i,p}(\xi)w_i}{\sum_j N_{j,p}(\xi)w_j}$$

Allows exact representation of conic shapes (circles, cylinders).

4. Control Points

Physical coordinates $\mathbf{P}_i$ that parameterize the geometry:

$$\mathbf{C}(\xi) = \sum_i R_{i,p}(\xi) \mathbf{P}_i$$

Read directly from CAD formats ('.stp', '.igs'), avoiding standard FE mesh generation.

B-Spline Basis Functions

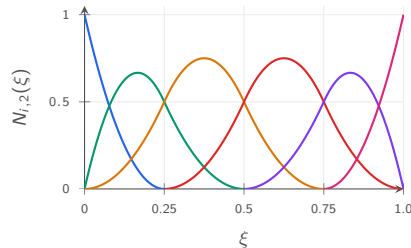
Over open knot vector $\Xi = \{\xi_0, \dots, \xi_m\}$, the **Cox-de Boor** recurrence:

$$N_{i,0}(\xi) = \begin{cases} 1 & \xi_i \leq \xi < \xi_{i+1} \\ 0 & \text{otherwise} \end{cases}$$

$$N_{i,p} = \frac{\xi - \xi_i}{\xi_{i+p} - \xi_i} N_{i,p-1} + \frac{\xi_{i+p+1} - \xi}{\xi_{i+p+1} - \xi_{i+1}} N_{i+1,p-1}$$

Key Properties:

- **Partition of Unity:** $\sum_i N_{i,p}(\xi) = 1 \quad \forall \xi$
- **Non-negativity:** $N_{i,p}(\xi) \geq 0 \quad \forall \xi, i$
- **Local Support:** $N_{i,p}(\xi) = 0$ outside $[\xi_i, \xi_{i+p+1})$



Degree- $p = 2$ basis, $\Xi = \{0, 0, 0, 0.25, 0.5, 0.75, 1, 1, 1\}$

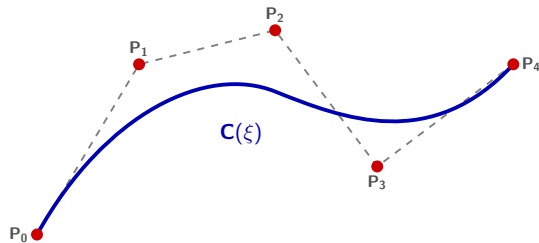
B-Spline Curve Concept

A B-spline/NURBS curve $\mathbf{C}(\xi)$ is defined as:

$$\mathbf{C}(\xi) = \sum_{i=0}^{n-1} R_{i,p}(\xi) \mathbf{P}_i$$

- $\mathbf{P}_i$ represent the **control points**.
- The control points form a piecewise linear **control polygon**.
- Lack of Kronecker delta property: $\mathbf{C}(\xi_i) \neq \mathbf{P}_i$ in general, except at the clamped boundary ends.

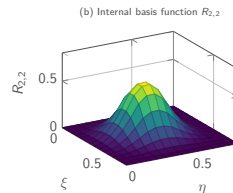
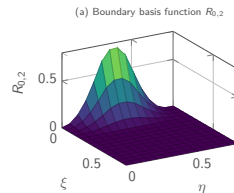
[Interactive IGA Curve Web App](#)



2D Bivariate Basis Functions

Basis Function Shapes:

- Bivariate basis shape depends on location in control net.
- **Boundary Basis** $R_{0,2}(\xi, \eta)$:
Edge-supported, peaks at boundaries.
- **Internal Basis** $R_{2,2}(\xi, \eta)$: Fully internal support, peaks at center of element grid.



3D surface plot of boundary vs. internal bivariate B-spline functions

2D Bivariate IGA Mapping

Tensor-Product NURBS:

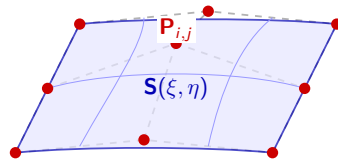
- Bivariate rational basis functions $R_{i,j}^{p,q}(\xi, \eta)$ extend 1D B-splines to 2D surfaces:

$$R_{i,j}^{p,q}(\xi, \eta) = \frac{N_{i,p}(\xi)M_{j,q}(\eta)w_{i,j}}{\sum_k \sum_l N_{k,p}(\xi)M_{l,q}(\eta)w_{k,l}}$$

- Geometric mapping $\mathbf{S}(\xi, \eta)$ maps the parametric square $[0, 1]^2$ to the physical 2D domain:

$$\mathbf{S}(\xi, \eta) = \sum_i \sum_j R_{i,j}^{p,q}(\xi, \eta) \mathbf{P}_{i,j}$$

- $\mathbf{P}_{i,j}$ forms a bidimensional control net lattice.



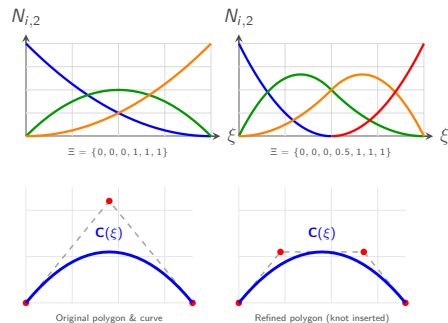
2D B-spline surface patch mapping concept

h-Refinement (Knot Insertion)

Knot Insertion: Adding elements by subdividing knot spans.

- **Why:** Localize refinement near high-stress areas (e.g., notches).
- **How:** Boehm's algorithm inserts new values $\bar{\xi}$ into Ξ .
- **Pros:** Geometry is preserved exactly. Continuity is maintained (C^{p-1}).
- **Cons:** Increases mesh size (number of elements)
→ larger system matrix → slower solver.

Interactive *h*-Refinement App

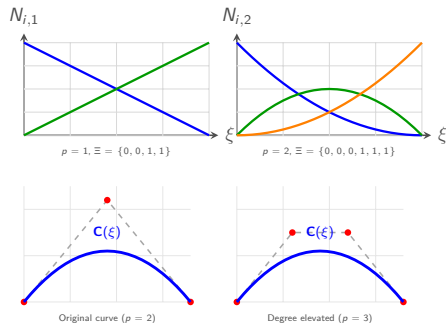


p -Refinement (Degree Elevation)

Degree Elevation: Increasing polynomial degree of basis functions.

- **Why:** Boost accuracy for smooth solutions and wave dynamics.
- **How:** Linear operator elevates degree $p \rightarrow p + \Delta p$, increasing knot multiplicity to preserve continuity.
- **Pros:** Keeps geometric shape identical. Avoids high-order Runge oscillations of Lagrange FEM.
- **Cons:** Wider basis support increases matrix bandwidth and solver overhead.

Interactive p -Refinement App



k -Refinement & Exact Geometry

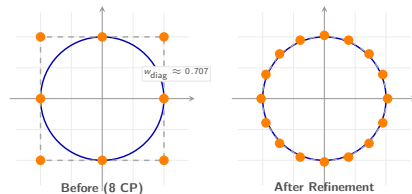
k -Refinement: Elevating polynomial degree while maintaining maximum inter-element continuity.

- **Why:** Achieve maximum accuracy per DOF. High continuity (C^{p-1}) is optimal for dynamics.
- **How:** Elevate degree first ($p \rightarrow p + \Delta p$), *then* insert internal knots.
- **Pros:** Unique to IGA. Keeps DOFs minimal for a given error tolerance.
- **Cons:** Higher computational assembly stencil (wider support).

Interactive k -Refinement App

Continuity Comparison

Method	Sequence	Continuity
$p \rightarrow h$	Insert first	C^0 (FEA-like)
k -ref	Elevate first	$C^{p+\Delta p-1}$



Exact circle: diagonal weights $w_i = \frac{\sqrt{2}}{2}$. Knot insertion converges net to curve.

CHAPTER 4

Simulation: Full Order Model

Governing Dynamic Equations

To simulate the dynamic response we resolve the semi-discrete system:

Equation of Motion

$$\mathbf{M}(\mu) \ddot{\mathbf{U}}(t) + \mathbf{C}(\mu) \dot{\mathbf{U}}(t) + \mathbf{K}(\mu) \mathbf{U}(t) = \lambda(t) \mathbf{F}_{\text{ext}}$$

$$\text{with initial conditions: } \mathbf{U}(0) = \mathbf{U}_0, \quad \dot{\mathbf{U}}(0) = \mathbf{V}_0$$

Mass bilinear form:

$$m(\mathbf{v}_h, \ddot{\mathbf{u}}_h; \mu) = \int_{\Omega(\mu)} \rho \mathbf{v}_h \cdot \ddot{\mathbf{u}}_h d\Omega$$

Global mass matrix:

$$\mathbf{M}_{ab}(\mu) = m(\mathbf{R}_a, \mathbf{R}_b; \mu)$$

Rayleigh damping: $\mathbf{C}(\mu) = \alpha_M \mathbf{M}(\mu) + \beta_K \mathbf{K}(\mu)$

Stiffness bilinear form:

$$k(\mathbf{u}_h, \mathbf{v}_h; \mu) = \int_{\Omega(\mu)} \boldsymbol{\varepsilon}(\mathbf{v}_h)^T \mathbf{D} \boldsymbol{\varepsilon}(\mathbf{u}_h) d\Omega$$

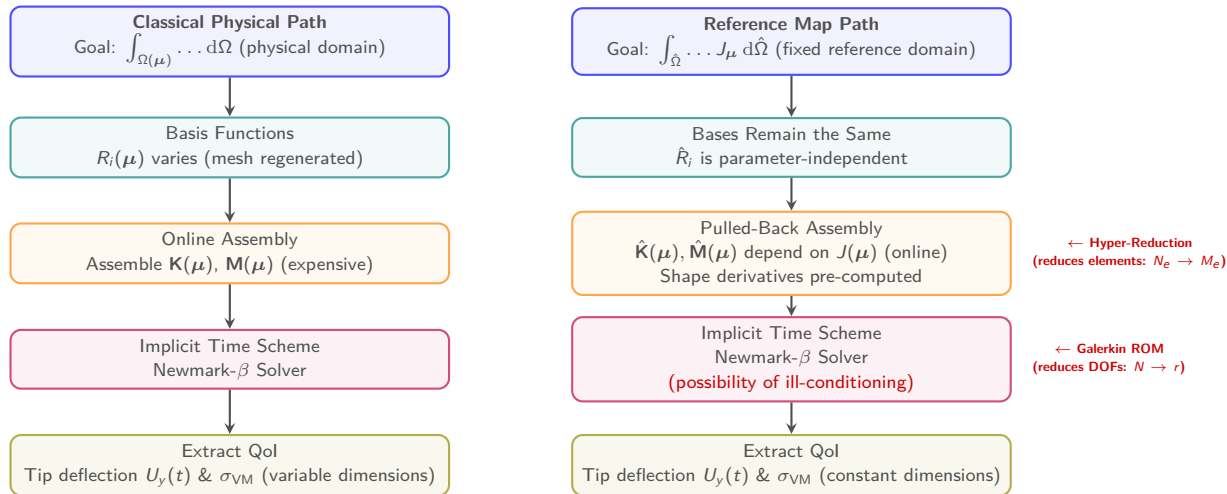
Global stiffness matrix:

$$\mathbf{K}_{ab}(\mu) = k(\mathbf{R}_b, \mathbf{R}_a; \mu)$$

Online Assembly Bottleneck

As μ changes, the integration domain $\Omega(\mu)$ shifts $\Rightarrow$ full mesh re-assembly is required at each time step.

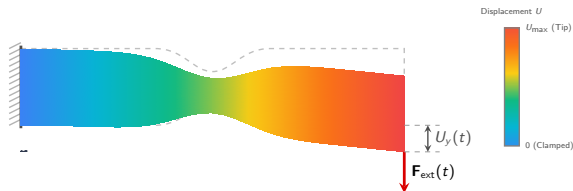
Simulation Path & Solver Workflow



Model Displacement Field

Deflection Contours:

- Represents the physical displacement magnitude across the 2D surface.
- Transitions smoothly from **zero** at the clamped boundary ($x = 0$) to a **maximum** at the loaded free tip ($x = L$).
- **Quantity of Interest (QoI)**: Tip vertical deflection $U_y(t)$ is measured at the free edge.



Model Stress & Von Mises Definition

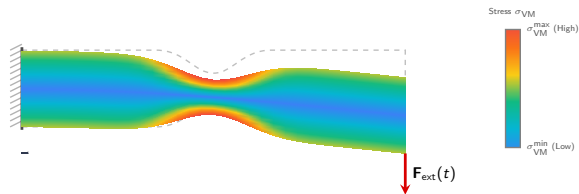
Stress Reconstruction:

- **Key Relation:** Once the displacement field $\mathbf{u}$ is solved, we evaluate strain $\varepsilon(\mathbf{u})$, constitutive stress $\boldsymbol{\sigma} = \mathbf{D} : \varepsilon$, and reconstructed yield criteria.

- **Von Mises Stress σ_{VM} :**

$$\sigma_{VM} = \sqrt{\sigma_{xx}^2 - \sigma_{xx}\sigma_{yy} + \sigma_{yy}^2 + 3\tau_{xy}^2}$$

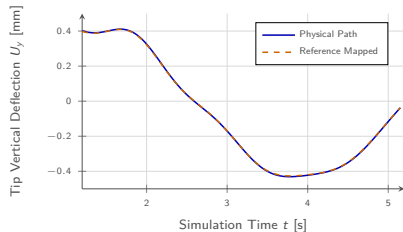
- **Stress Concentration:** Localized at notch throats (mechanical fatigue risk).



Physical vs. Reference Mapped Path: Validation

Numerical Validation:

- Compare the transient vertical deflection $U_y(t)$ computed via both pathways.
- **Exact Agreement:** Near-zero difference validates the mathematical consistency of the pullback transformation.



Tip vertical deflection $U_y(t)$ transient curve

Mesh Convergence Study

A dyadic grid convergence study was conducted under static tip bending load using the interactive simulation application to justify the choice of the 80×40 elements mesh baseline.

Resolution	Elements	DoFs	Tip Disp. U_y (mm)	Relative Error
10×5	50	168	3.538	16.51%
20×10	200	528	3.889	8.24%
40×20	800	1,848	4.083	3.66%
80×40	3,200	6,888	4.185	1.24%
160×80	12,800	26,568	4.238	Reference

The 80×40 discretization is selected as the operating baseline because:

- Refining to 160×80 elements yields a change of only 1.24% in displacement.
- Avoids a $3.8\times$ increase in equations ($6,888 \rightarrow 26,568$ DOFs).

CHAPTER 5

Reduced Order Modeling, Hyper-Reduction & Digital Twin

Why ROM? (Motivation)

The Full-Order Bottleneck

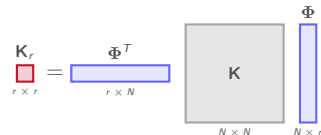
- High-fidelity Full Order Model (FOM) has $N = 6,564$ DOFs.
- Online matrix assembly scales with mesh size $\mathcal{O}(N)$.
- Solves are too slow for real-time interactivity.

Need for Interactivity

- Interactive client-side web browser (Digital Twin).
- Must bypass the full-order assembly bottleneck.
- Target frame rate: 20–60 FPS (< 1.5 ms solve time).

Interactive Budget

Online Solve Time: < 1.5 ms



	FOM	ROM
Dimension	$N = 6,564$	$r = 15$
Matrix entries	$\approx 43\text{M}$	225

What Do We Need to Achieve It? (The Steps)

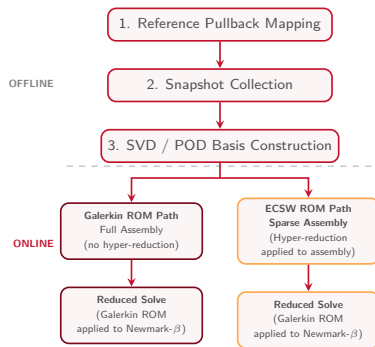
Building a shape-parametric Reduced Order Model involves:

Offline Preparation

- 1. **Pullback**: Map to reference $\hat{\Omega}$.
- 2. **Snapshots**: Generate state vectors.
- 3. **SVD / POD**: Build the basis Φ .

Online Interactivity

- Project onto the basis and solve in real-time ($r \ll N$).



Step 1: Reference Pullback Mapping

Diffeomorphic Pullback to Static $\hat{\Omega}$ (Thesis Section 3.4)

Coordinate transformation maps varying physical coordinates $\mathbf{x} \in \Omega(\mu)$ to static reference coordinates $\hat{\mathbf{x}} \in \hat{\Omega}$:

$$\nabla_{\mathbf{x}} \mathbf{u} = \mathbf{J}_{\xi}^{-T}(\xi; \mu) \nabla_{\xi} \hat{\mathbf{u}}, \quad d\Omega = J_{\xi, \mu}(\xi) d\hat{\Omega}$$

where $\mathbf{J}_{\xi}(\xi; \mu) = \frac{\partial \mathbf{x}}{\partial \xi}$ is the coordinate Jacobian, and $J_{\xi, \mu}(\xi) = \det \mathbf{J}_{\xi}$ is the mapping weight.

State Projection on Static Basis (Φ)

The discrete state vector $\mathbf{U}(\mu)$ is parameter-dependent, but projected onto static POD modes:

$$\mathbf{U}(\mu) \approx \Phi \mathbf{q}(\mu)$$

Why this matters:

- **Static Modes:** The SVD projection matrix Φ is constant and computed offline.
- **Online Efficiency:** Only the low-dimensional state vector $\mathbf{q}(\mu) \in \mathbb{R}^r$ needs to be resolved online.

The Parameter Dependency ($\mathbf{K}(\mu)$)

Stiffness bilinear form maps parameter μ into the spatial integral:

$$\hat{k}(\mathbf{u}, \delta \mathbf{u}; \mu) = \int_{\hat{\Omega}} \left(\mathbf{J}_{\xi}^{-T} \nabla_{\xi} \mathbf{u} \right) : \mathbf{D} : \left(\mathbf{J}_{\xi}^{-T} \nabla_{\xi} \delta \mathbf{u} \right) J_{\xi, \mu} d\hat{\Omega}$$

Why it matters:

- Traps μ inside the integral.
- Standard ROM cannot precompute $\tilde{\mathbf{u}}^T \mathbf{K}(\mu) \tilde{\mathbf{u}}$.
- Requires $O(N)$ full online assembly.

Step 2: Snapshot Collection (Data Generation)

What is a Snapshot?

- A high-fidelity state vector representing the structural displacement field $\mathbf{U}(t) \in \mathbb{R}^N$ at a given time or parameter set.

Compilation of snap matrix

- Compile all state vectors into snapshot matrix:

$$\mathbf{S}_u = [\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_d] \in \mathbb{R}^{N \times d}$$

Representative Load Cases

FOM simulations are run offline across representative configurations to capture dominant physics:

- Tip bending load
- Mid-span bending load
- Analytical axial tension

Step 3: SVD / POD Basis Construction

Dimensionality Reduction:

- Perform Singular Value Decomposition (SVD):

$$\mathbf{S}_u = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$$

- Keep the first r dominant modes of $\mathbf{U}$ to form the Reduced Basis matrix $\mathbf{\Phi} \in \mathbb{R}^{N \times r}$:

$$\mathbf{U}(t) \approx \mathbf{\Phi} \mathbf{q}(t), \quad r \ll N$$

POD Convergence Metrics

Modes r	Error $\epsilon(r)$	Energy $\eta(r)$
1	2.34%	99.945%
5	0.89%	99.992%
10	0.11%	99.999%

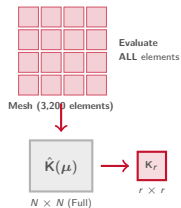
$\epsilon(r) = \sqrt{1 - \eta(r)}$ decays rapidly, reaching high accuracy at just $r = 10$ modes.

Step 4: Galerkin Projection vs. ECSW Hyper-reduction

Galerkin Projection ($r \times r$)

$$\mathbf{M}_r \ddot{\mathbf{q}} + \mathbf{C}_r \dot{\mathbf{q}} + \mathbf{K}_r(\mu) \mathbf{q} = \mathbf{F}_r(t)$$

Assembly Bottleneck: Requires assembling the full matrix $\hat{\mathbf{K}}(\mu) \in \mathbb{R}^{N \times N}$ online. No online speedup ($0.92\times$).

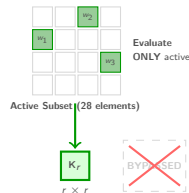


ECSW Hyper-Reduction

ECSW integrates only over a sparse active subset of elements $\tilde{\Omega} \subset \Omega$:

$$\mathbf{K}_{r,\text{ECSW}}(\mu) = \sum_{e \in \tilde{\Omega}} w_e \Phi_e^T \mathbf{K}_e(\mu) \Phi_e$$

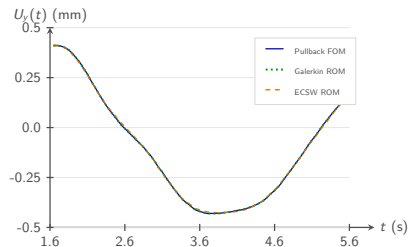
Bypasses the assembly bottleneck, evaluating only 28 out of 3,200 elements online.



Step 5: Real-Time Solver & Benchmarks

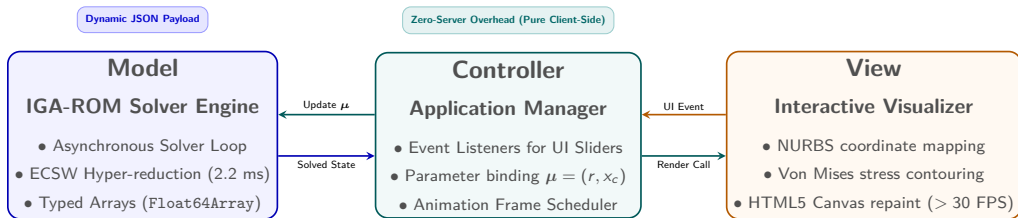
Quantitative Comparative Performance

Solver Strategy	Cells	L_2 Error	Time	Speedup
Full Order (FOM)	3,200	Baseline	142.1 ms	1.0×
Galerkin ROM	3,200	0.012%	154.5 ms	0.92×
ECSW ROM	28	0.054%	2.2 ms	64.5×



Tip deflection $U_y(t)$ comparison curves

Client-Side Software Architecture (Digital Twin)



CHAPTER 6

Conclusion & Future Perspectives

Summary of Achievements & Implemented Methods

Key Achievements

- **High-Fidelity Solver:** Built a 2D Isogeometric Analysis (IGA) physical solver ($N = 6,564$ DOFs).
- **Hyper-Reduction:** Deployed ECSW on only 28 cells, yielding a $64.5\times$ **speedup** with 0.054% error.
- **Digital Twin:** Interactive client-side JS application achieving real-time solve times (< 1.5 ms).

Implemented Methods & References

- **POD / SVD:** Dimension reduction basis.
Ref: Benner et al. (2015)
- **ECSW Hyper-Reduction:** Sparse mesh quadrature.
Ref: Farhat et al. (2014)
- **Diffeomorphic Pullback:** Shape-parameter mapping preserving mesh topology.
- **Newmark- β :** Transient dynamic solver.

Alternatives & Future Perspectives

Other Methods (Unexplored)

- **DEIM / MDEIM:** Discrete Empirical Interpolation for nonlinear terms.
Refs: Chaturantabut (2010), Bonomi (2017)
- **Gappy POD:** Regression-based state reconstruction.
- **Machine Learning:** Autoencoder-based nonlinear ROMs or PINNs.
- **Multi-Field ROMs:** Damage-plasticity decompositions.
Ref: Kehls (2025)

Future Research Paths

- **Nonlinear Kinematics:** Large displacements using Green-Lagrange strain directly in reduced space.
- **3D NURBS:** Trivariate solid formulations and multi-patch assemblies.
- **Adaptive Bases:** Online local basis update/enrichment (e.g., Kriging).

Thank You

Questions & Discussion